Declaration

```
template<typename Clock,typename Duration>
cv_status wait_until(
    std::unique_lock<std::mutex>& lock,
    std::chrono::time_point<Clock,Duration> const& absolute_time);
```

Preconditions

`lock.owns_lock()` is `true`, and the lock is owned by the calling thread.

Effects

Atomically unlocks the supplied `lock` object and block until the thread is woken by a call to `notify_one()` or `notify_all()` by another thread, or `Clock::now()` returns a time equal to or later than `absolute_time` or the thread is woken spuriously. The `lock` object is locked again before the call to `wait_until()` returns.

Returns

`std::cv_status::no_timeout` if the thread was woken by a call to `notify_one()` or `notify_all()` or a spurious wakeup, `std::cv_status::timeout` otherwise.

Throws

`std::system_error` if the effects can't be achieved. If the `lock` object is unlocked during the call to `wait_until()`, it's locked again on exit, even if the function exits via an exception.

Note

The spurious wakeups mean that a thread calling `wait_until()` may wake even though no thread has called `notify_one()` or